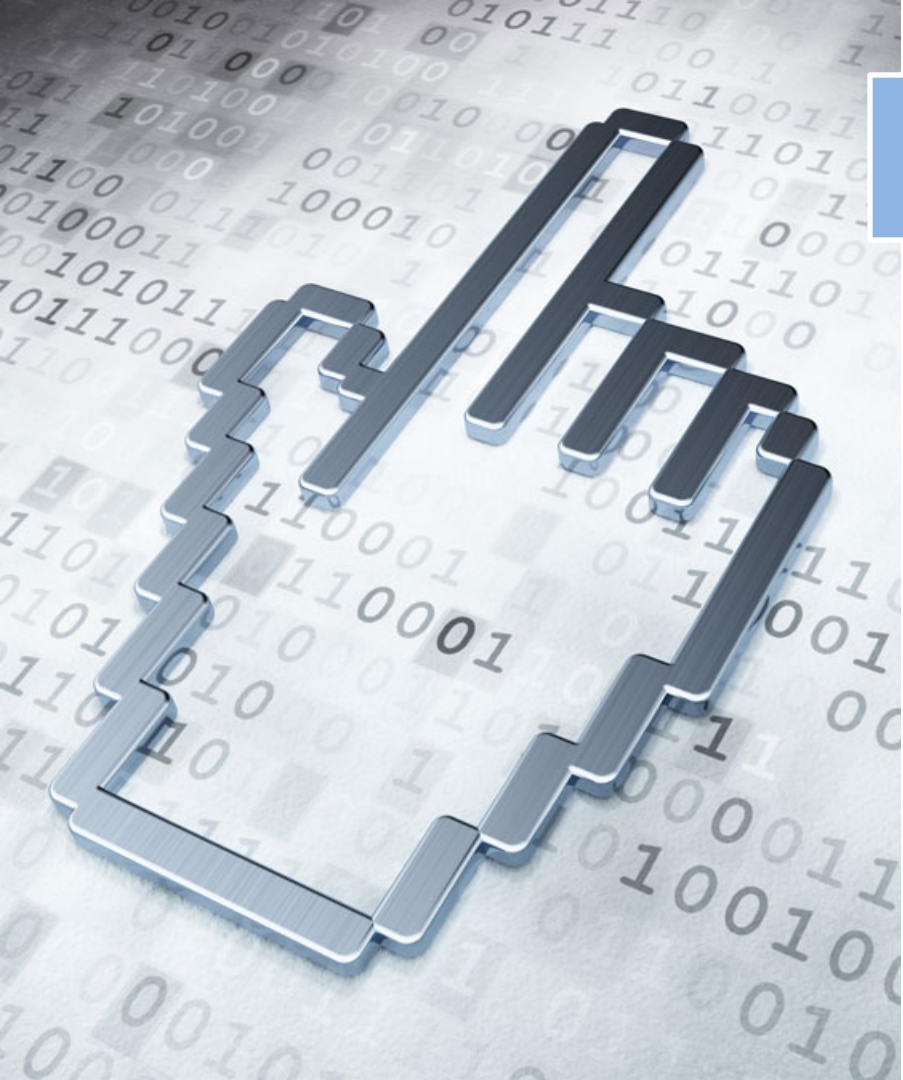




编译原理 第二章 语言及其文法



哈尔滨工业大学 陈鄞 单丽莉



本章内容

2.1 基本概念

2.2 文法的定义

2.3 语言的定义

2.4 文法的分类

2.5 CFG的语法分析树

➤ **串** (*String*)

➤ 串是一个**有穷符号** (*symbol*) **序列**

➤ 符号可以使用单字符表示，也可以使用多个字符表示：

➤ 字母、数字、标点符号、...

➤ WHILE、ID、IDS, ...

➤ 串 (*String*)

➤ 串是一个有穷符号 (*symbol*) 序列

➤ 串 s 的长度, 通常记作 $|s|$, 是指 s 中符号的个数

➤ 例: $|abcb|=4$

➤ 空串 (*empty string*) 是长度为0的串, 用 ε (*epsilon*) 表示

➤ $|\varepsilon|=0$

- 如果 x 和 y 是串，那么 x 和 y 的**连接**(concatenation)是把 y 附加到 x 后面而形成的**串**，记作 xy
- 例如，如果 $x=dog$ 且 $y=house$ ，那么 $xy=doghouse$
- **空串** 是连接运算的**单位元**(identity)，即，对于任何串 s 都有， $\varepsilon s = s\varepsilon = s$

设 y, z 是串，若 $x=yz$ ，则称 y 是 x 的**前缀**，
 z 是 x 的**后缀**。

➤ 串 s 的幂运算

$$\begin{cases} s^0 = \varepsilon, \\ s^n = s^{n-1}s, n \geq 1 \end{cases}$$

➤ $s^1 = s^0 s = \varepsilon s = s, \quad s^2 = ss, \quad s^3 = sss, \quad \dots$

➤ 例：如果 $s = ba$,

那么 $s^1 = ba, \quad s^2 = baba, \quad s^3 = bababa, \quad \dots$

➤ 字母表 Σ 是一个有穷符号集合

例：

➤ 八进制数码表：{ 0, 1, 2, 3, 4, 5, 6, 7 }

➤ *Unicode* 字符集

➤ 英文的单词集

字母表上的运算

➤ 字母表 Σ_1 和 Σ_2 的 **乘积** (*product*)

$$\Sigma_1 \Sigma_2 = \{ab \mid a \in \Sigma_1, b \in \Sigma_2\}$$

例： $\{0, 1\} \{a, b\} = \{0a, 0b, 1a, 1b\}$

➤ $\Sigma_1 \Sigma_2$ 是所有符合条件的长度为2的串的集合

字母表上的运算

➤ 字母表 Σ_1 和 Σ_2 的乘积(*product*)

➤ 字母表 Σ 的 n 次幂(*power*)

$$\begin{cases} \Sigma^0 = \{ \varepsilon \} \\ \Sigma^n = \Sigma^{n-1} \Sigma, n \geq 1 \end{cases}$$

例： $\{0, 1\}^3 = \{0, 1\} \{0, 1\} \{0, 1\}$
 $= \{000, 001, 010, 011, 100, 101, 110, 111\}$

字母表的 n 次幂：字母表上所有长度为 n 的串构成的集合

字母表上的运算

- 字母表 Σ_1 和 Σ_2 的**乘积**(*product*)
- 字母表 Σ 的 **n 次幂**(*power*)
- 字母表 Σ 的**正闭包**(*positive closure*)
 - $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

例： $\{a, b, c, d\}^+ = \{a, b, c, d,$
 $aa, ab, ac, ad, ba, bb, bc, bd, \dots,$
 $aaa, aab, aac, aad, aba, abb, abc, \dots\}$

字母表的正闭包：字母表上**所有长度大于0的串**构成的集合

- 字母表 Σ_1 和 Σ_2 的**乘积**(*product*)
- 字母表 Σ 的 **n 次幂**(*power*)
- 字母表 Σ 的**正闭包**(*positive closure*)
- 字母表 Σ 的**克林闭包**(*Kleene closure*)
 - $\Sigma^* = \Sigma^0 \cup \Sigma^+ = \Sigma^0 \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

例： $\{a, b, c, d\}^* = \{\epsilon, a, b, c, d, aa, ab, ac, ad, ba, bb, bc, bd, \dots, aaa, aab, aac, aad, aba, abb, abc, \dots\}$

字母表的克林闭包：字母表上所有串（包括空串）构成的集合

➤ 设 Σ 是一个字母表, $\forall L \subseteq \Sigma^*$, L 称为字母表 Σ 上的一个语言 (Language), $\forall x \in L$, x 叫做 L 的一个句子。

例: 字母表 $\{0, 1\}$ 上的语言

$\{0, 1\}$

$\{0, 1, 00, 11, 01, 10\}$

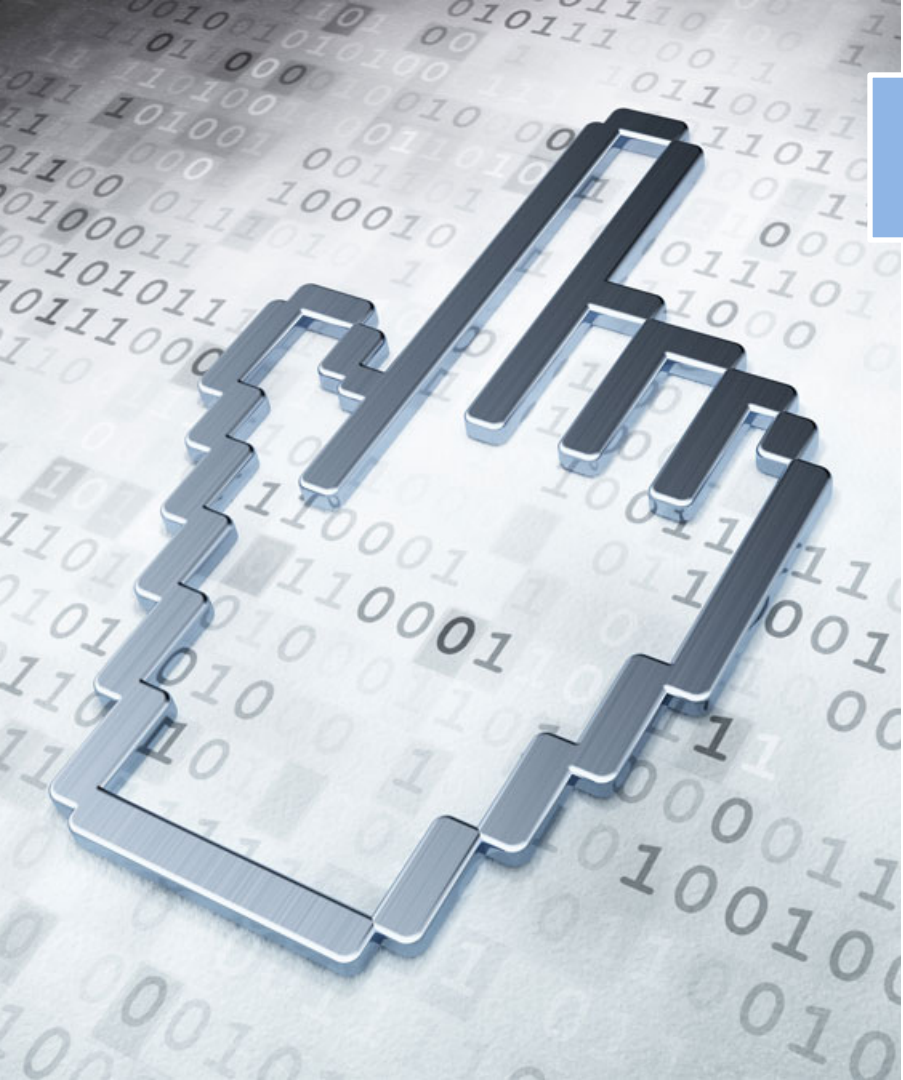
$\{00, 11\}^*$

思考: 程序设计语言的字母表是什么?

程序设计语言是什么样的句子的集合?

如何描述这些句子构成的规则?

利用有限的语法规则表示无限程序的集合。



提纲

2.1 基本概念

2.2 文法的定义

2.3 语言的定义

2.4 文法的分类

2.5 CFG的语法分析树

2.2 文法的定义

➤ 自然语言的例子——句子的构成规则

- <句子> → <名词短语> <动词短语>
- <名词短语> → <形容词> <名词短语>
- <名词短语> → <名词>
- <动词短语> → <动词> <名词短语>
- <形容词> → *little* | *pretty*
- <名词> → *boy* | *girl*
- <名词> → *apple* | *pie*
- <动词> → *eats* | *watches*

没有尖括号的部分表示**语言**
的基本符号(字母表符号)

尖括号括起来部分称为**语法成分**

文法的形式化定义——语言的有穷描述

$$G = (V_T, V_N, P, S)$$

➤ V_T : 终结符集合

终结符 (*terminal symbol*) 是文法所定义的语言的基本符号(字母表符号), 有时也称为 *token*。

➤ 例: $V_T = \{ \textit{apple, boy, eats, little, pretty, girl, pie, watches} \}$

文法的形式化定义

$$G = (V_T, V_N, P, S)$$

➤ V_T : 终结符集合

➤ V_N : 非终结符集合

非终结符(*nonterminal*) 是用来表示**语法成分**的符号，也称为“**语法变量**”

➤ 例: $V_N = \{ \langle \text{句子} \rangle, \langle \text{名词短语} \rangle, \langle \text{动词短语} \rangle, \langle \text{名词} \rangle, \langle \text{动词} \rangle, \langle \text{形容词} \rangle \}$

文法的形式化定义

$$G = (V_T, V_N, P, S)$$

➤ V_T : 终结符集合

$$V_T \cap V_N = \Phi$$

有穷符号集合

➤ V_N : 非终结符集合

➤ P : 产生式集合

$V_T \cup V_N$: 称为G的文法符号集

产生式(*production*)描述了将**终结符**和**非终结符**组合成**串**的方法, 即语法成分的构成方法。

产生式的一般形式: $\alpha \rightarrow \beta$, 读作: α 定义为 β

➤ $\alpha \in (V_T \cup V_N)^+$, 且 α 中至少包含一个 V_N 中的元素: 称为产生式的**头**(*head*) 或**左部**(*left side*)

➤ $\beta \in (V_T \cup V_N)^*$: 称为产生式的**体**(*body*)或**右部**(*right side*)

文法的形式化定义

$$G = (V_T, V_N, P, S)$$

➤ V_T : 终结符集合

➤ V_N : 非终结符集合

➤ P : 产生式集合

产生式(*production*)描述了将终结符和非终结符组合成串的方法，即语法成分的构成方法。

➤ 例： $P = \left\{ \begin{array}{l} \langle \text{句子} \rangle \rightarrow \langle \text{名词短语} \rangle \langle \text{动词短} \rangle, \\ \langle \text{名词短语} \rangle \rightarrow \langle \text{形容词} \rangle \langle \text{名词短语} \rangle, \\ \dots \end{array} \right\}$

文法的形式化定义

$$G = (V_T, V_N, P, S)$$

➤ V_T : 终结符集合

➤ V_N : 非终结符集合

➤ P : 产生式集合

➤ S : 开始符号

$S \in V_N$ 。 **开始符号** (*start symbol*) 表示的是该文法中最大的语法成分

➤ 例: $S = \langle \text{句子} \rangle$

文法的形式化定义

$$G = (V_T, V_N, P, S)$$

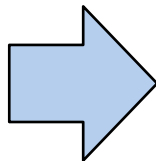
- V_T : 终结符集合
- V_N : 非终结符集合
- P : 产生式集合
- S : 开始符号

约定

可以只用产生式集合表示文法G,
第一个产生式的头就是开始符号。

例: $G = (\{ \text{id}, +, *, (,) \}, \{ E \}, P, E)$

$P = \{ E \rightarrow E + E,$
 $E \rightarrow E * E,$
 $E \rightarrow (E),$
 $E \rightarrow \text{id} \}$



$G : E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow (E)$
 $E \rightarrow \text{id}$

产生式的简写

➤ 对一组具有相同左部的 α 产生式

$$\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$$

可以简记为：

$$\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

读作： α 定义为 β_1 ，或者 β_2 ，...，或者 β_n 。 $\beta_1, \beta_2, \dots, \beta_n$ 称为 α 的候选式(Candidate)。

➤ 例

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

$$E \rightarrow \text{id}$$



$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$$

➤ 下述符号是**终结符**

- (*a*) 英文字母表中**排在前面**的小写字母，如 *a*、*b*、*c*
- (*b*) **运算符**，如 +、*等
- (*c*) **标点符号**，如括号、逗号等
- (*d*) **数字**0、1、...、9
- (*e*) **粗体字符串**，如id、if等

➤ 下述符号是**非终结符**

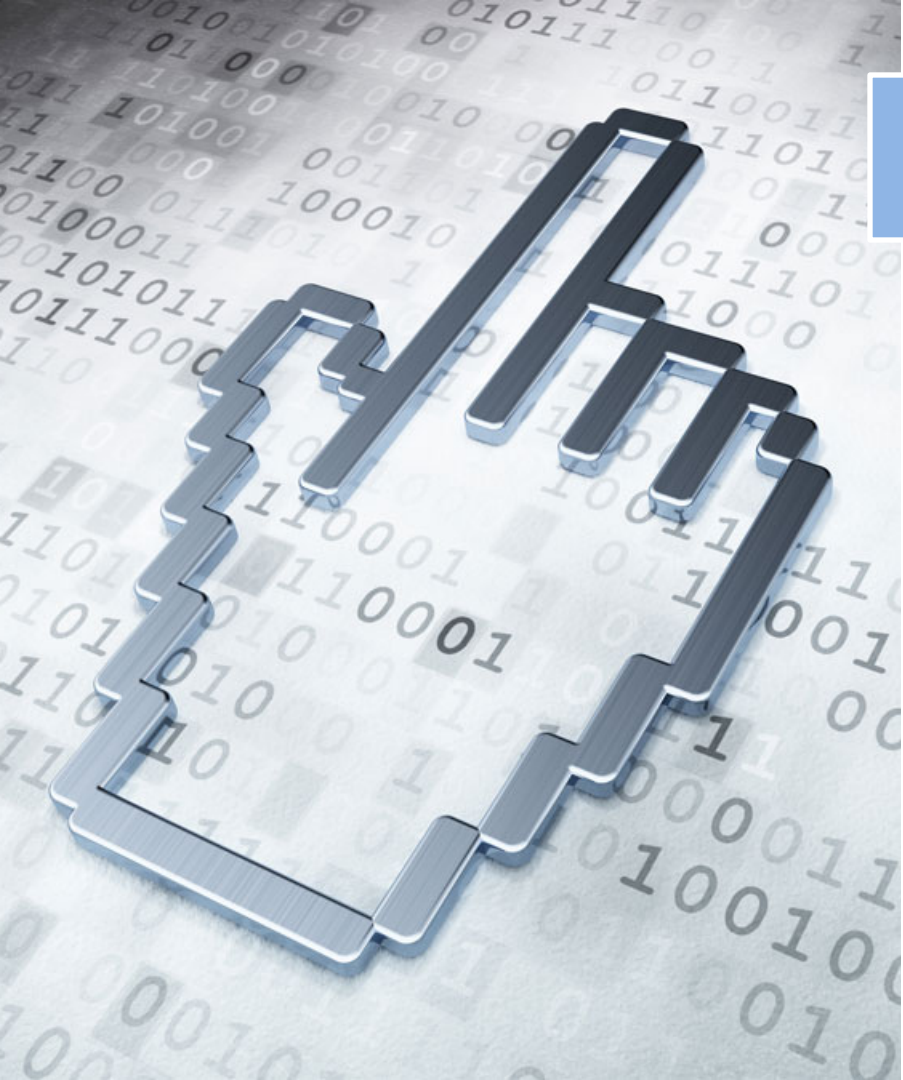
(a) 字母表中**排在前面的大写字母**，如 A 、 B 、 C

(b) 字母 S ，通常表示开始符号

(c) **小写、斜体的名字**，如 $expr$ 、 $stmt$ 等

(d) **代表程序构造的大写字母**。如 E (表达式)、 T (项)和
 F (因子)

- 字母表中排在大写字母后面的大写字母（如 X 、 Y 、 Z ）
表示**文法符号**（即终结符或非终结符）
- 字母表中排在小写字母后面的小写字母（主要是 u 、 v 、 $\dots$ 、 z ）
表示**终结符号串**（包括空串）
- 小写希腊字母，如 α 、 β 、 γ ，表示**文法符号串**（包括空串）



提纲

2.1 基本概念

2.2 文法的定义

2.3 语言的定义

2.4 文法的分类

2.5 CFG的语法分析树

2.3 语言的定义

自然语言文法的例子：

- ① <句子> → <名词短语> <动词短语>
- ② <名词短语> → <形容词> <名词短语>
- ③ <名词短语> → <名词>
- ④ <动词短语> → <动词> <名词短语>
- ⑤ <形容词> → *little* | *pretty*
- ⑥ <名词> → *boy* | *girl*
- ⑦ <名词> → *apple* | *pie*
- ⑧ <动词> → *eats* | *watches*

有了文法，如何判定一个词串是否是满足文法规则的合法句子？

单词串： *little boy eats apple*

➤ 推导:

给定文法 $G=(V_T, V_N, P, S)$, 如果 $\alpha \rightarrow \beta \in P$, 那么可以将符号串 $\gamma\alpha\delta$ 中的 α 替换为 β , 也就是说, 将 $\gamma\alpha\delta$ 重写 (rewrite) 为 $\gamma\beta\delta$, 记作 $\gamma\alpha\delta \Rightarrow \gamma\beta\delta$ 。此时, 称文法中的符号串 $\gamma\alpha\delta$ 直接推导 (directly derive) 出 $\gamma\beta\delta$

➤ 简而言之, 就是用产生式的右部替换产生式的左部

推导 (Derivations) 和 归约 (Reductions)

例: *little boy eats apple*

文法:

- ① <句子> → <名词短语> <动词短语>
- ② <名词短语> → <形容词> <名词短语>
- ③ <名词短语> → <名词>
- ④ <动词短语> → <动词> <名词短语>
- ⑤ <形容词> → *little* | *pretty*
- ⑥ <名词> → *boy* | *girl*
- ⑦ <名词> → *apple* | *pie*
- ⑧ <动词> → *eats* | *watches*

归约: 归约是推导的逆过程

<句子> ⇒ <名词短语> <动词短语>
⇒ <形容词> <名词短语> <动词短语>
⇒ little <名词短语> <动词短语>
⇒ little <名词> <动词短语>
⇒ little boy <动词短语>
⇒ little boy <动词> <名词短语>
⇒ little boy eats <名词短语>
⇒ little boy eats <名词>
⇒ little boy eats apple

推导

归约

- 如果 $\alpha_0 \Rightarrow \alpha_1, \alpha_1 \Rightarrow \alpha_2, \alpha_2 \Rightarrow \alpha_3, \dots, \alpha_{n-1} \Rightarrow \alpha_n$, 则可以记作 $\alpha_0 \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \alpha_3 \Rightarrow \dots \Rightarrow \alpha_{n-1} \Rightarrow \alpha_n$, 称符号串 α_0 经过 n 步推导出 α_n , 可简记为 $\alpha_0 \xRightarrow{n} \alpha_n$
- $\alpha \xRightarrow{0} \alpha$
- $\xRightarrow{+}$ 表示“经过至少一步推导”
- $\xRightarrow{*}$ 表示“经过若干（包括0）步推导”

最左推导和最右推导

➤ 最左推导(Left-most Derivation)

- 每次推导都选择句型最左边的非终结符展开。

——与最右归约对应

➤ 最右推导(Right-most Derivation)

- 每次推导都选择句型最右边的非终结符展开。

——与最左归约对应。

➤有了文法（语法规则），如何判定某一词串是否是满足文法规则的？

➤**推导**（派生）——从**生成**语言的角度

➤从文法的开始符号开始，经过若干步推导，能够推导出该词串。

➤**归约**——从**识别**语言的角度

➤从该词串出发，经过若干步归约，能够归约出文法的开始符号。

- 如果 $S \xRightarrow{*} \alpha$, $\alpha \in (V_T \cup V_N)^*$, 则称 α 是 G 的一个 **句型** (*sentential form*)
 - 一个句型中既可以包含 **终结符**, 又可以包含 **非终结符**, 也可能是 **空串**
- 如果 $S \xRightarrow{*} w$, $w \in V_T^*$, 则称 w 是 G 的一个 **句子** (*sentence*)
 - 句子是 **不包含非终结符的句型**

例

<句子>⇒ <名词短语> <动词短语>

⇒ <形容词> <名词短语> <动词短语>

⇒ *little* <名词短语> <动词短语>

⇒ *little* <名词> <动词短语>

⇒ *little boy* <动词短语>

⇒ *little boy* <动词> <名词短语>

⇒ *little boy eats* <名词短语>

⇒ *little boy eats* <名词>

句子 → ⇒ *little boy eats apple*

句型

➤ 由文法 G 的开始符号 S 推导出的所有句子构成的集合称为**文法 G 生成的语言**，记为 $L(G)$ 。即

$$L(G) = \{ w \mid S \xRightarrow{*} w, w \in V_T^* \}$$

* 思考：文法 $G: E \rightarrow E+E \mid E^*E \mid (E) \mid \text{id}$

语言 $L(G)$ 中包含多少个句子？

➤ 设有两文法 G_1 和 G_2 ，如果 $L(G_1) = L(G_2)$ ，则称 G_1 和 G_2 为**等价文法**。

➤ 文法 G

$$① \quad S \rightarrow L \mid LT$$

$$② \quad T \rightarrow L \mid D \mid TL \mid TD$$

$$③ \quad L \rightarrow a \mid b \mid c \mid \dots \mid z$$

$$④ \quad D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid \dots \mid 9$$

课后练习:请写出无符号
整数或浮点数的文法

$$T \Rightarrow \underline{TL}$$

$$\Rightarrow \underline{TDL}$$

$$\Rightarrow \underline{TDDL}$$

$$\Rightarrow \underline{TLDDL}$$

...

$$\Rightarrow \underline{TD} \dots LDDL$$

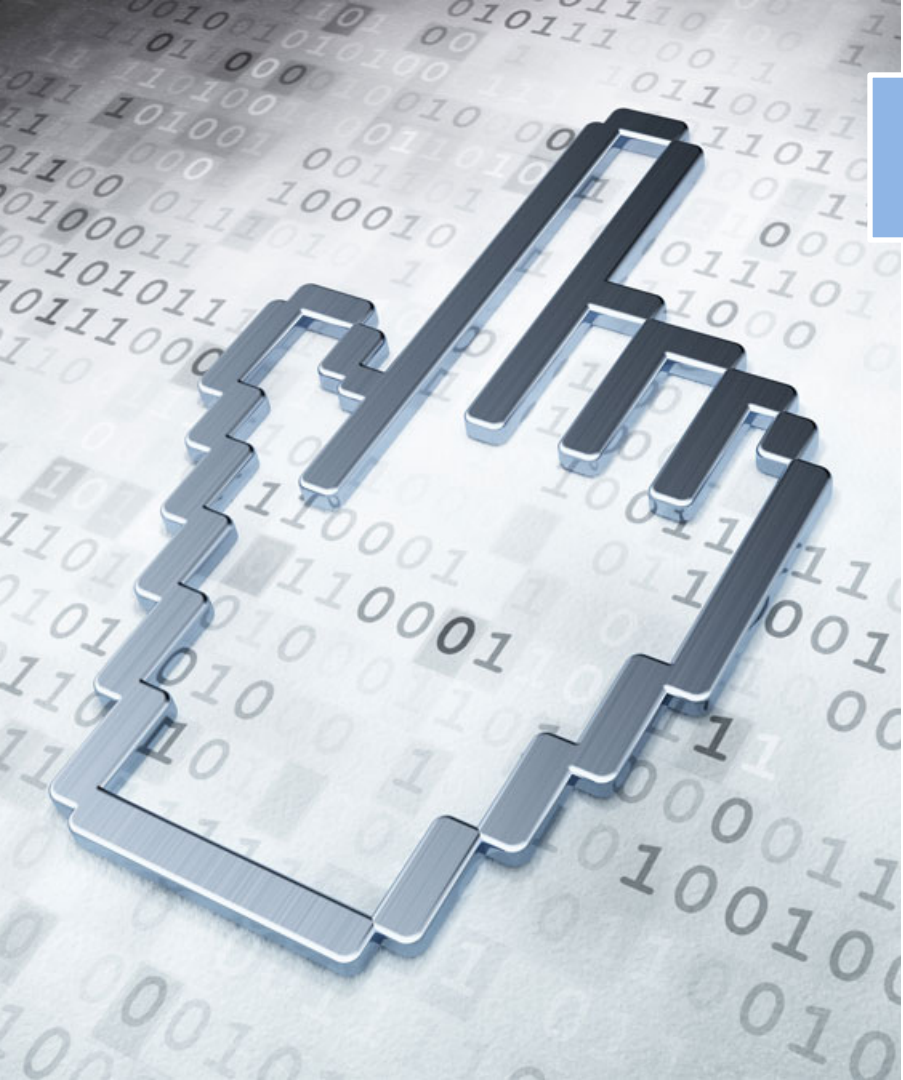
$$\Rightarrow \underline{DD} \dots LDDL$$

该文法识别的串:

由字母开头的字母数字串

运算	定义和表示
L 和 M 的并	$L \cup M = \{s \mid s \text{ 属于 } L \text{ 或者 } s \text{ 属于 } M\}$
L 和 M 的连接	$LM = \{st \mid s \text{ 属于 } L \text{ 且 } t \text{ 属于 } M\}$
L 的幂	$\begin{cases} L^0 = \{ \varepsilon \} \\ L^n = L^{n-1}L, n \geq 1 \end{cases}$
L 的Kleene闭包	$L^* = \bigcup_{i=0}^{\infty} L^i$
L 的正闭包	$L^+ = \bigcup_{i=1}^{\infty} L^i$

例：令 $L=\{a, b, \dots, z\}$, $D=\{0, 1, \dots, 9\}$ 。则 $L(L \cup D)^*$ 表示的语言是
字母开头的字母数字串的集合



提 纲

2.1 基本概念

2.2 文法的定义

2.3 语言的定义

2.4 文法的分类

2.5 CFG的语法分析树

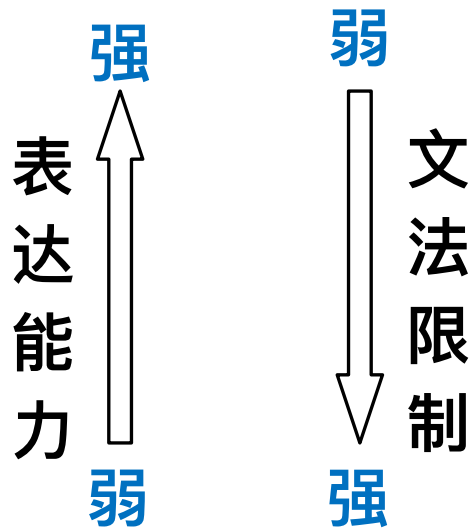
➤ 乔姆斯基(*Chomsky*)文法分类体系

➤ 0型文法 (*Type-0 Grammar*)

➤ 1型文法 (*Type-1 Grammar*)

➤ 2型文法 (*Type-2 Grammar*)

➤ 3型文法 (*Type-3 Grammar*)



$$\alpha \rightarrow \beta$$

➤ **无限制文法**(*Unrestricted Grammar*) / **短语结构文法**

(*Phrase Structure Grammar, PSG*)

➤ $\forall \alpha \rightarrow \beta \in P, \quad \alpha \in (V_T \cup V_N)^+, \text{ 且 } \alpha \text{ 中至少包含1个非终结符, } \beta \in (V_T \cup V_N)^*$

➤ **0型语言**

➤ 由0型文法 G 生成的语言 $L(G)$

1型文法 (Type-1 Grammar)

$$\alpha \rightarrow \beta$$

➤ 上下文有关文法 (Context-Sensitive Grammar, CSG)

➤ $\forall \alpha \rightarrow \beta \in P, \quad |\alpha| \leq |\beta|$

➤ 产生式的一般形式: $\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2 \quad (\beta \neq \varepsilon)$

➤ 上下文有关语言 (1型语言)

➤ 由上下文有关文法 (1型文法) G 生成的语言 $L(G)$

可以包含 $S \rightarrow \varepsilon$, 但此时 S 不能出现在任何产生式的右部。

例 上下文有关语言的文法CSG

$L = \{ a^n b^n c^n \mid n > 0 \}$ 的文法

$S \rightarrow aBC \mid aSBC$

$CB \rightarrow BC$

$aB \rightarrow ab$

$bB \rightarrow bb$

$bC \rightarrow bc$

$cC \rightarrow cc$

“可以证明”不存在CFG G , 使 $L(G)=L$

2型文法 (*Type-2 Grammar*)

$$\alpha \rightarrow \beta$$

➤ 上下文无关文法 (*Context-Free Grammar, CFG*)

➤ $\forall \alpha \rightarrow \beta \in P, \alpha \in V_N$

➤ 产生式的一般形式: $A \rightarrow \beta, A \in V_N, \beta \in (V_T \cup V_N)^*$

例:

$$S \rightarrow L \mid LT$$

$$T \rightarrow L \mid D \mid TL \mid TD$$

$$L \rightarrow a \mid b \mid c \mid d \mid \dots \mid z$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid \dots \mid 9$$

2型文法 (*Type-2 Grammar*)

$$\alpha \rightarrow \beta$$

➤ 上下文无关文法 (*Context-Free Grammar, CFG*)

➤ $\forall \alpha \rightarrow \beta \in P, \alpha \in V_N$

➤ 产生式的一般形式: $A \rightarrow \beta, A \in V_N, \beta \in (V_T \cup V_N)^*$

➤ 上下文无关语言 (2型语言)

➤ 由上下文无关文法 (2型文法) G 生成的语言 $L(G)$

通常使用上下文无关文法来描述程序设计语言的语法结构

3型文法 (Type-3 Grammar)

$$\alpha \rightarrow \beta$$

➤ **正则文法** (Regular Grammar, RG)

设 $A, B \in V_N$, $w \in V_T^+$

➤ **右线性** (Right Linear) **文法**: $A \rightarrow wB$ 或 $A \rightarrow w$

➤ **左线性** (Left Linear) **文法**: $A \rightarrow Bw$ 或 $A \rightarrow w$

➤ **左线性文法和右线性文法等价** 都称为正则文法

例 (右线性文法)

① $S \rightarrow a \mid b \mid c \mid d$

② $S \rightarrow aT \mid bT \mid cT \mid dT$

③ $T \rightarrow a \mid b \mid c \mid d \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5$

④ $T \rightarrow aT \mid bT \mid cT \mid dT \mid 0T \mid 1T \mid 2T \mid 3T \mid 4T \mid 5T$

文法 G (上下文无关文法)

① $S \rightarrow L \mid LT$

② $T \rightarrow L \mid D \mid TL \mid TD$

③ $L \rightarrow a \mid b \mid c \mid d$

④ $D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5$

3型文法 (Type-3 Grammar)

$$\alpha \rightarrow \beta$$

➤ **正则文法** (Regular Grammar, RG) 设 $A, B \in V_N, w \in V_T^+$

➤ **右线性** (Right Linear) **文法**: $A \rightarrow wB$ 或 $A \rightarrow w$

➤ **左线性** (Left Linear) **文法**: $A \rightarrow Bw$ 或 $A \rightarrow w$

➤ **正则语言** (3型语言) 左线性文法和右线性文法等价, 都称为正则文法

➤ 由正则文法 (3型文法) G 生成的语言 $L(G)$

正则文法能描述程序设计语言的多数单词

四种文法之间的关系是将产生式作递进式限制而定义的。

四种文法之间的逐级“包含”关系如下：



不同的文法可以定义相同的语言，称为等价文法。

➤ 程序设计语言的有些语言规则不能用上下文无关文法描述

例2.9 $L_1 = \{ wcw \mid w \in \{a, b\}^+ \}$

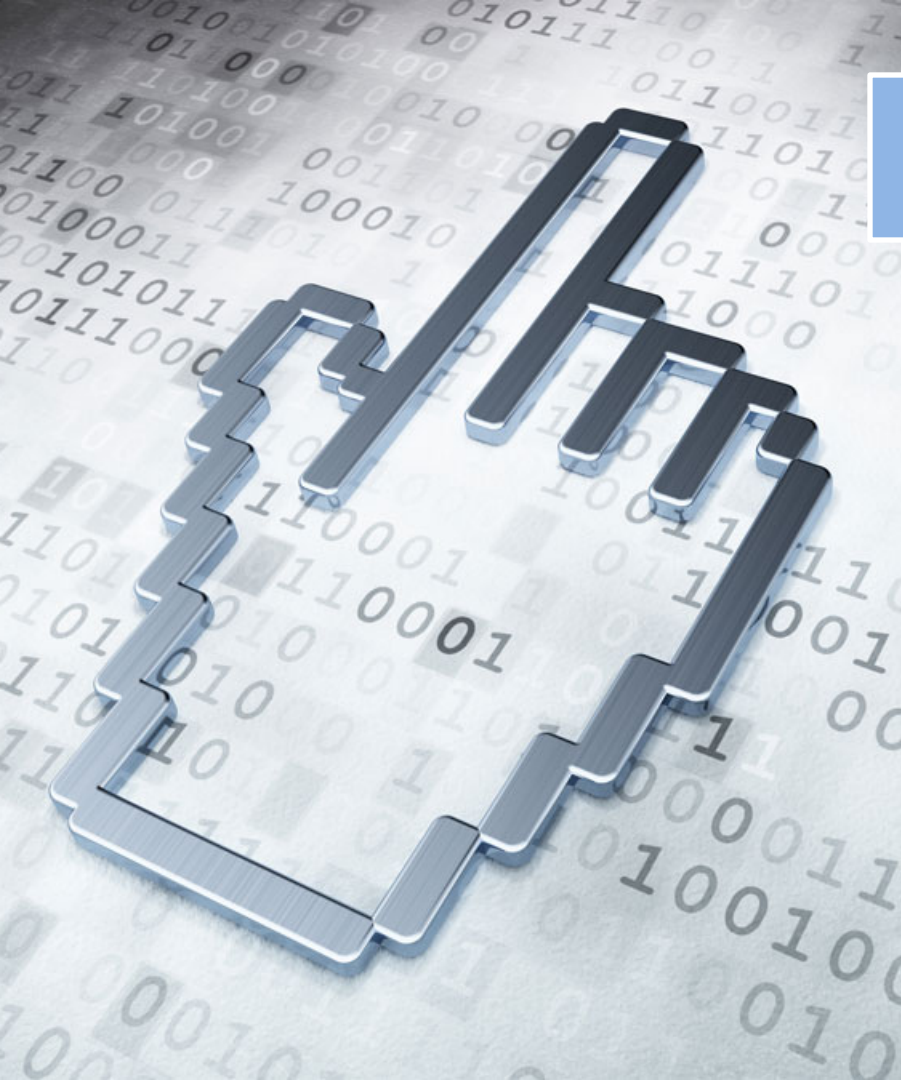
aabcaab 就是 L_1 的一个句子

- 检查程序中标识符的声明应先于引用的抽象

例2.10 $L_2 = \{ a^n b^m c^n d^m \mid n, m \geq 0 \}$

- 检查过程声明的形参个数和过程引用的参数个数是否一致问题的抽象

上下文无关文法无法描述的语义规则检查由语义分析阶段完成



提纲

2.1 基本概念

2.2 文法的定义

2.3 语言的定义

2.4 文法的分类

2.5 CFG的语法分析树

最左推导: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$

最右推导: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(E+id) \Rightarrow -(id+id)$

文法:

① $E \rightarrow E + E$

② $E \rightarrow E * E$

③ $E \rightarrow -E$

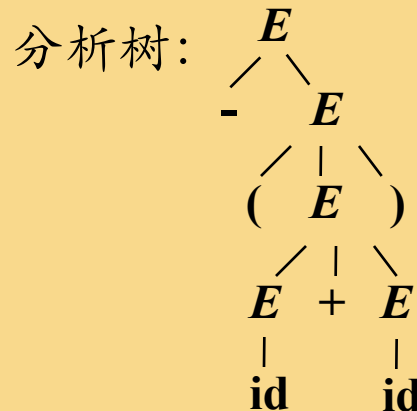
④ $E \rightarrow (E)$

⑤ $E \rightarrow id$

句型:

$-(id+id)$

- 最左推导和最右推导最终的分析树是一样的
- 分析树忽略了推导的次序



- 叶结点的标号既可以是**非终结符**, 也可以是**终结符**。从左到右排列叶节点得到的符号串称为这棵**树的产出**(*yield*)或**边缘**(*frontier*)

(句型的) 短语

- 给定一个句型，其分析树中的每一棵**子树的边缘**称为该句型的一个**短语**(*phrase*)
- 如果子树只有**父子两代结点**，那么这棵子树的边缘称为该句型的一个**直接短语**(*immediate phrase*)

文法:

① $E \rightarrow E + E$

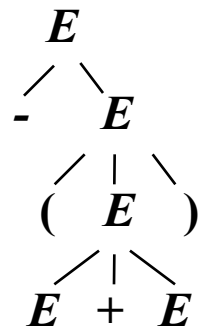
② $E \rightarrow E * E$

③ $E \rightarrow - E$

④ $E \rightarrow (E)$

⑤ $E \rightarrow \text{id}$

分析树:



句型:

➤ $-(E+E)$

短语:

➤ $-(E+E)$

➤ $(E+E)$

➤ $E+E$

直接短语:

➤ $E+E$

句型的**短语**一定是句型的**子串**

直接短语**一定**是某产生式的**右部**

直接短语是归约动作的候选子串

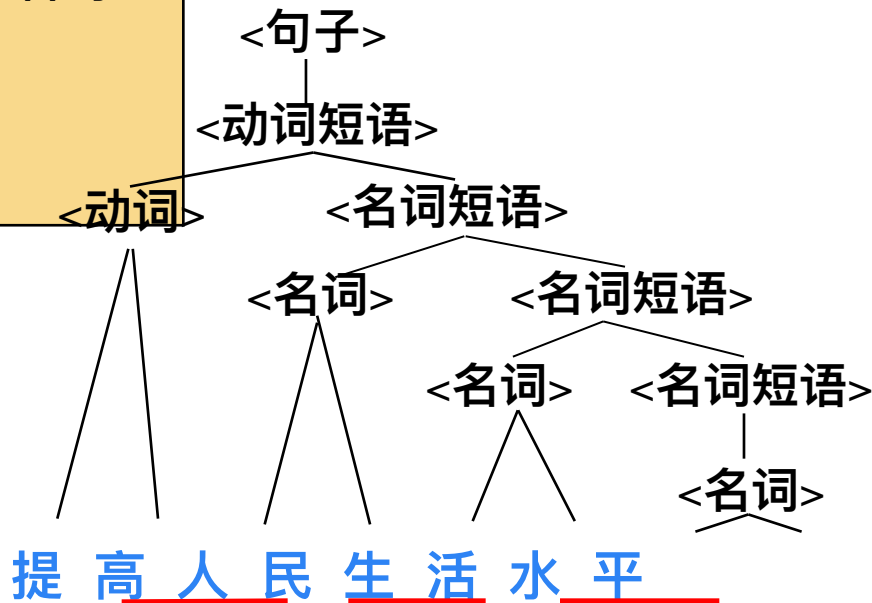
文法:

- ①<句子> → <动词短语>
- ②<动词短语> → <动词> <名词短语>
- ③<名词短语> → <名词> <名词短语> | <名词>
- ④<动词> → 提 高
- ⑤<名词> → 高 人 | 人 民 | 民 生 | 生 活
| 活 水 | 水 平

输入: 提高人民生活水平

准确地找到直接短语,
才能准确地进行归约

但若某产生式的右部是某个句型的子串, 它却不一定是给定句型的直接短语



➤ 如果一个文法可以为某个句子生成**多棵分析树**，则称这个文法是**二义性的**

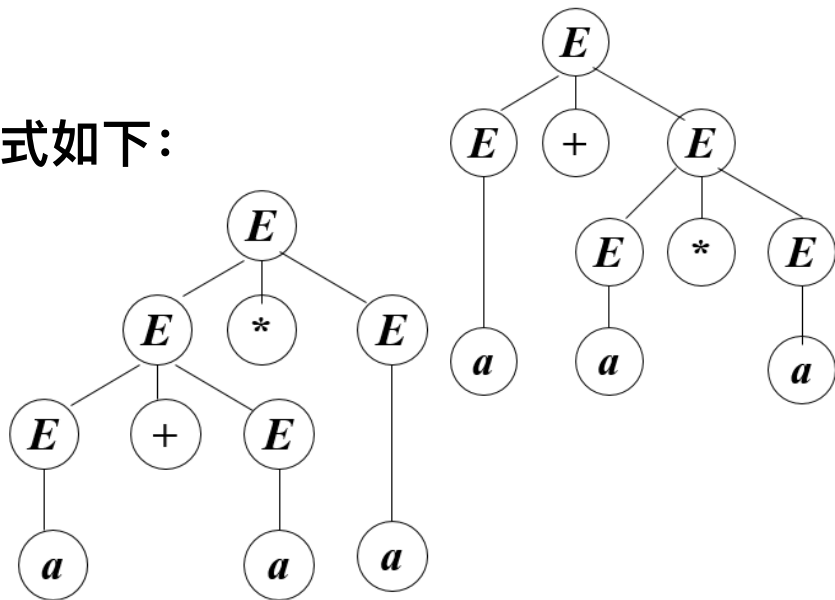
例：考虑表达式下面的文法 $G[E]$, 其产生式如下：

$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

对于句子 $a + a * a$, 有如下两个最左推导：

$$\begin{aligned} E &\Rightarrow \underline{E} + E \Rightarrow \underline{a} + E \Rightarrow a + \underline{E * E} \\ &\Rightarrow a + \underline{a * E} \Rightarrow a + a * \underline{a} \end{aligned}$$

$$\begin{aligned} E &\Rightarrow \underline{E * E} \Rightarrow \underline{E} + E * E \Rightarrow \underline{a} + E * E \\ &\Rightarrow a + \underline{a * E} \Rightarrow a + a * \underline{a} \end{aligned}$$



- 产生二义性的原因：该文法未指定加法和乘法的运算顺序
- 二义性带来的问题：同一算式运算结果不同。

例

➤ 文法

➤ $S \rightarrow \text{if } E \text{ then } S$

| $\text{if } E \text{ then } S \text{ else } S$

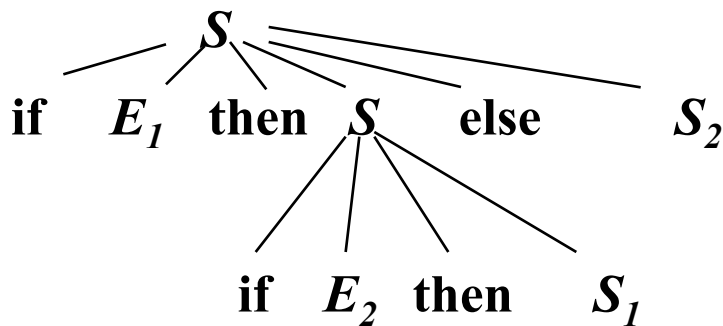
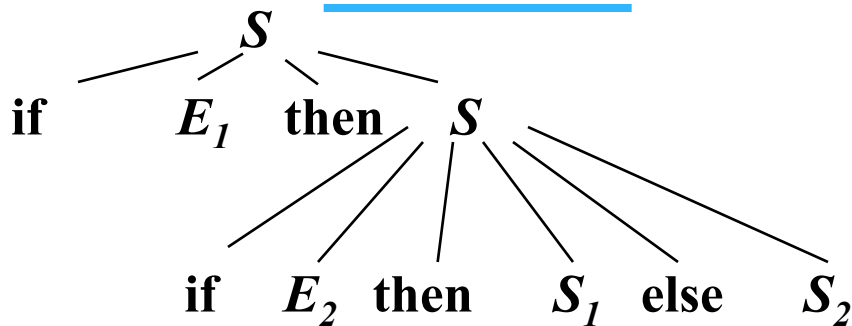
| other

} 条件语句

其他语句

➤ 句型

➤ $\text{if } E_1 \text{ then } \underline{\text{if } E_2 \text{ then } S_1 \text{ else } S_2}$



- 对于存在二义性的句子，语法分析时需要确定选择哪一种语法分析树，即消除二义性。

改造文法，细化规则表达。

描述算术运算的无二义性文法 G :

$$(1) E \rightarrow E+T$$

$$(2) E \rightarrow T$$

$$(3) T \rightarrow T*F$$

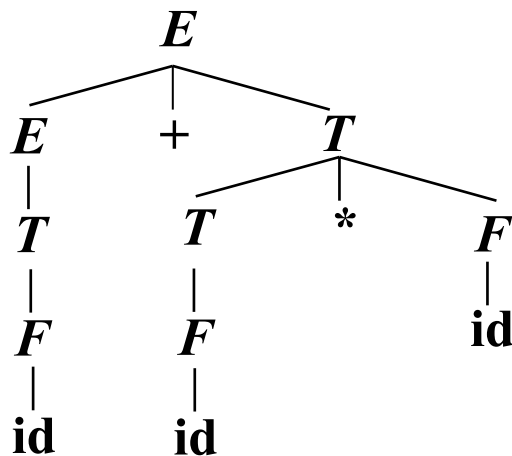
$$(4) T \rightarrow F$$

$$(5) F \rightarrow (E)$$

$$(6) F \rightarrow \text{id}$$

规则：先乘除，后加法。

句子：id+id*id 的语法分析树



描述if语句的无二义性文法的产生式

$$S \rightarrow matched_s \mid unmatched_s$$
$$matched_s \rightarrow \text{if expr then } matched_s \text{ else } matched_s \mid \text{other}$$
$$unmatched_s \rightarrow \text{if expr then } S \\ \mid \text{if expr then } matched_s \text{ else } unmatched_s$$

- 会使得文法变得复杂，且不存在通用的方法能将任意二义性方法转换为非二义性文法。
- 因此，为了方便处理，也可以使用二义性文法+消歧规则。

消除二义性的第二种方法

➤ 文法

➤ $S \rightarrow \text{if } E \text{ then } S$

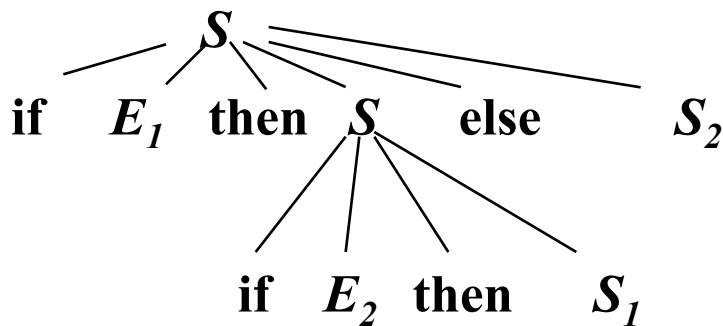
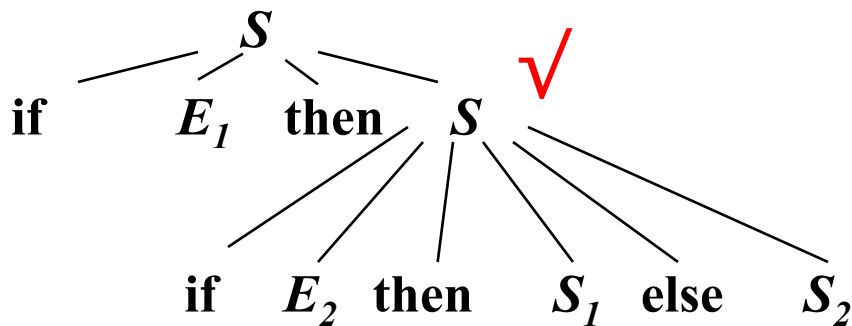
$\text{if } E \text{ then } S \text{ else } S$
 other

语法分析时加入消除二义性的规则

消歧规则：每个else和最近的尚未匹配的if匹配

➤ 句型

➤ $\text{if } E_1 \text{ then } \underline{\text{if } E_2 \text{ then } S_1 \text{ else } S_2}$



- 对于任意一个上下文无关文法，**不存在一个算法**，判定它是否是**无二义性**的；但能给出一组**充分条件**，满足这组充分条件的文法是**无二义性**的
- 确定性的上下文无关文法是无二义性的
- LL(k)、LR(k)文法是无二义性的
-

➤ 基本概念

➤ 字母表、字母表的正闭包和克林闭包、串、串的幂运算

➤ 文法的定义

➤ 形式化定义，终结符、非终结符、文法符号、产生式、符号串

➤ 语言的定义：形式化定义，推导与归约、语言与文法的关系

➤ 文法的分类：四种类型的文法定义及关系，四种类型语言

➤ CFG的分析树

➤ 句型分析树的构造、短语、直接短语、二义性文法

实验一 词法分析与语法分析

- 要求：实验指导书-词法分析与语法分析
- 实验准备：
 - 研读QQ群文件夹实验教程下的
 - 件_A_C--语言文法.pdf
 - 熟悉C--的词法和语法规则

- | | |
|-------------|-------|
| 1. 绪论 | (2学时) |
| 2. 语言及其文法 | (2学时) |
| 3. 词法分析 | (3学时) |
| 4. 语法分析 | (9学时) |
| 5. 语法制导翻译 | (6学时) |
| 6. 中间代码生成 | (7学时) |
| 7. 运行时的存贮组织 | (3学时) |
| 8. 代码优化 | (6学时) |
| 9. 代码生成 | (2学时) |



结束

